

6. Overview and concepts

Next  >



K-Means clustering

K-means clustering is a data mining method aiming to uncover hidden, natural groupings (i.e. clusters) in data. Data points belonging to the same cluster tend to share similar characteristics. They may correspond to such instances as customers with similar purchasing behaviour, plants belonging to the same species, or a subpopulation with a similar sociodemographic profile. Further, clusters discovered by k-means may also be used for prediction or classification.

K-means is well suited for exploratory data analysis and can be applied to various data types, including images and texts. This week, you will learn how to implement this data mining model in Python. You will also learn some standard techniques for optimising its clustering performance, in addition to visualising the discovered clusters.

Algorithm overview

The k-means clustering method iteratively finds k number of data points that serve as the **means** summary (hence the name "k-means") of all data points that belong to the same clusters. These data points are called **centroids**, and the value k must be chosen by the data analyst before implementing the model.

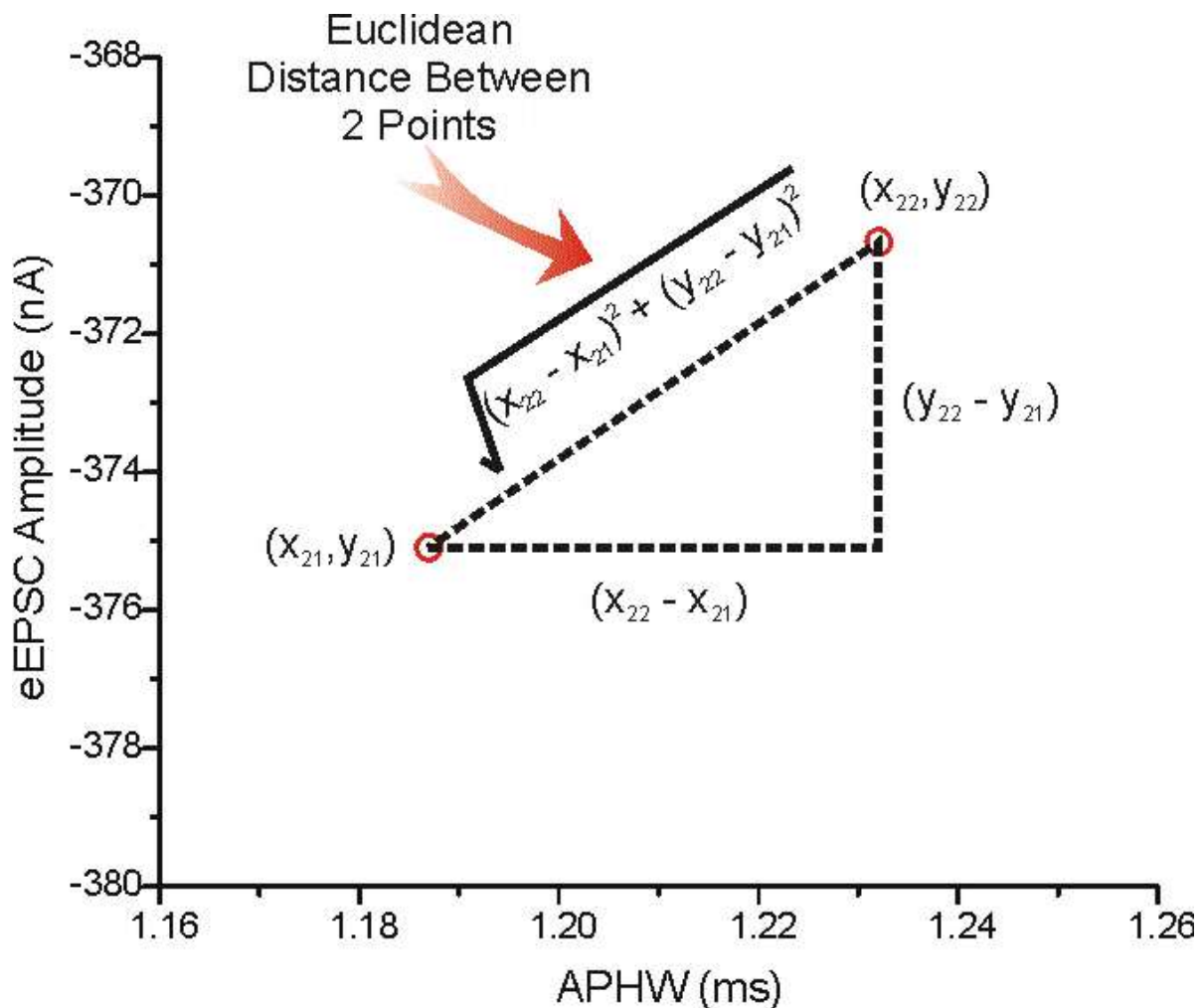
More specifically, k-means works according to the following steps:

1. k number of centroids are created randomly within the Euclidean space of a given dataset. There are several approaches for initially choosing the centroids. The k-means implementation in `scikit-learn` uses the **k-means++** initialisation method to achieve fast clustering.
2. Each data point in a dataset is then assigned to the nearest k centroids. The distance between a data point and a centroid is calculated as the **Euclidean distance** between them. In effect, this step minimises the total distance between all data points to their respective centroids.
3. k-means then recalculates the position of all k centroids by taking the means of attribute values of all data points belonging to each centroid.
4. Repeat Steps 2 and 3 until some criteria are met. These criteria include: the maximum number of iterations allowed, the minimum sum of distances between the data points and their corresponding centroid, or no significant changes to the positions of the centroids between two consecutive iterations.

As mentioned in Step 2 above, an essential equation in the k-means algorithm is the Euclidean distance d :

$$d(p,q) = \sqrt{\sum_{i=1}^n (p_i - q_i)^2}$$

, where p and q are two points in Euclidean n -dimensional space, and p_i and q_i are Euclidean vectors. The following illustration shows a 2-dimensional Euclidean space ($n=2$). Let point p be denoted by 2-dimensional coordinate (x_{21}, y_{21}) and point q be denoted by coordinate (x_{22}, y_{22}) . The Euclidean distance between the points is therefore the length of a simple straight line connecting the two points.



Source: hlab.stanford.edu (https://hlab.stanford.edu/brian/making_measurements.html).

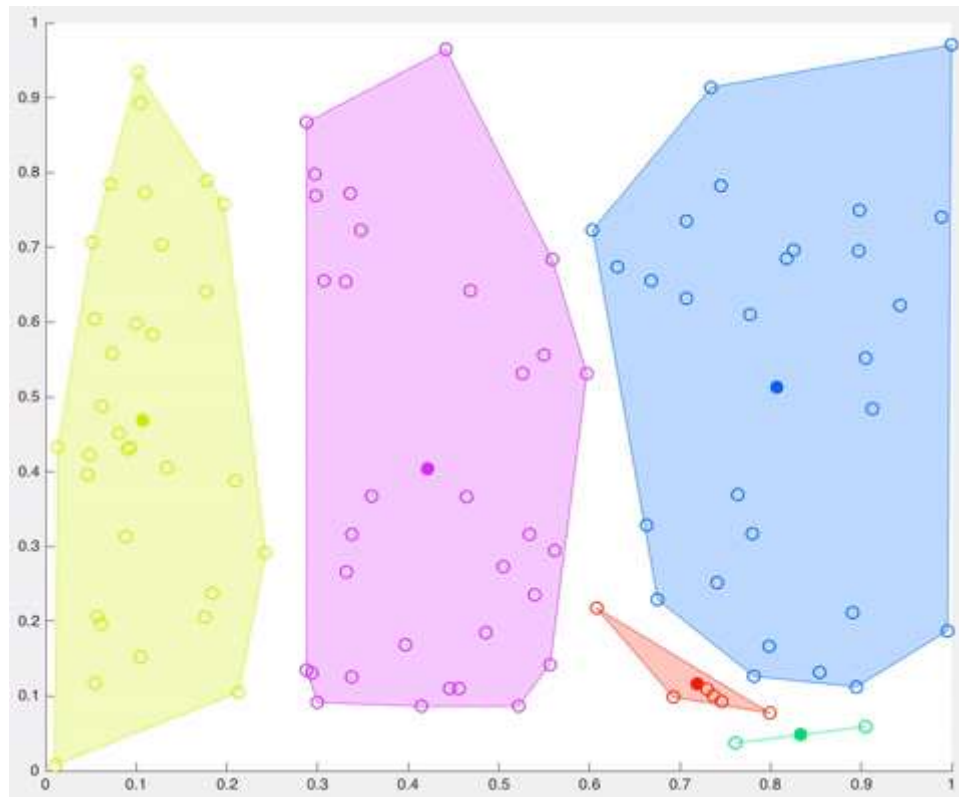
Here again, we encounter linear algebra. Note that the straight line between the two points above is a vector. The Euclidean distance is a mere adaptation of the L2 norm (<https://machinelearningmastery.com/vector-norms-machine-learning/#:~:text=The%20L2%20norm%20calculates%20the,is%20a%20positive%20distance%20value.>), of a vector in linear algebra.

The original L2 norm equation calculates the distance of a vector, let's say $x = (x_1, x_2, x_3)$, from the *origin* of the vector space (i.e. a vector of zero values):

$$\begin{aligned}
 |x| &= \sqrt{x_1^2 + x_2^2 + x_3^2} \\
 &= \sqrt{(x_1 - 0)^2 + (x_2 - 0)^2 + (x_3 - 0)^2} \\
 &= \sqrt{\sum_{i=1}^n (x_i - 0)^2} \\
 &= d(x, 0)
 \end{aligned}$$

Euclidean distance is a simple extension of the L2 norm, which replaces the *origin* with the coordinate of any given initial point.

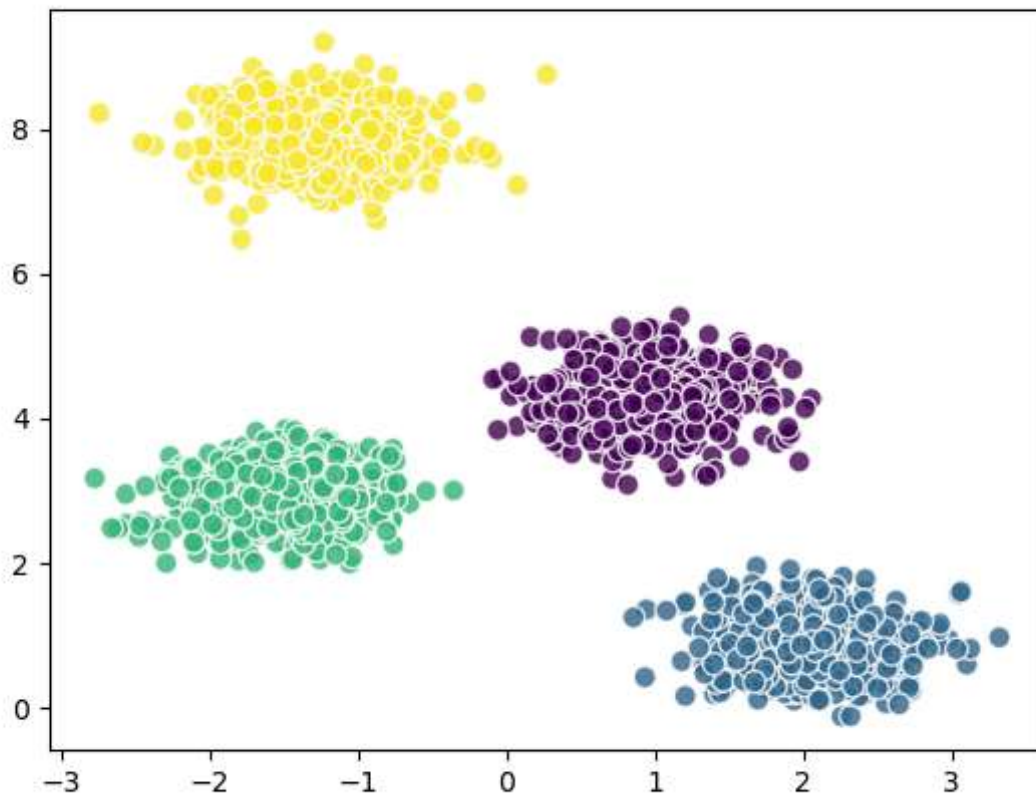
Below is a simulation of the effect after each k-means iteration. A cluster is represented by a colour-shaded convex hull. There are five clusters in this visualisation, each represented by a distinct colour. The centroid is the non-hollow data point at the centre of each cluster. After 11 iterations, we can observe that the position of each centroid no longer changes, suggesting an optimal clustering configuration has been reached and that the clustering process should terminate.



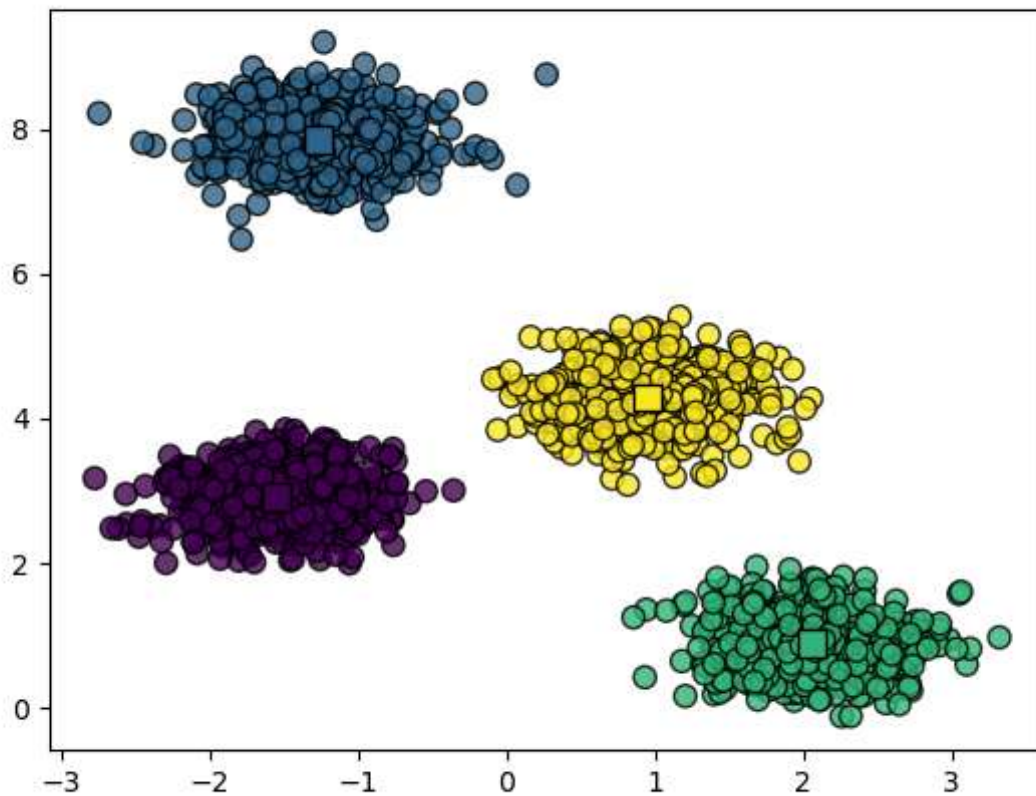
Source: KD Nuggets (<https://www.kdnuggets.com/2019/05/guide-k-means-clustering-algorithm.html#:~:text=K%2Dmeans%20allocates%20every%20data,centroid%20than%20any%20other%20centroid>).

Example 1

The first example below visualises a synthetic dataset with 4 naturally-occurring clusters of roughly spherical shapes.



Running k-means clustering ($k=4$) on the dataset shows four cluster centroids that correspond very well to the natural groupings of the data (in the image below, the centroids are the squares surrounded by other data points). This suggests that k-means produce good clusters.



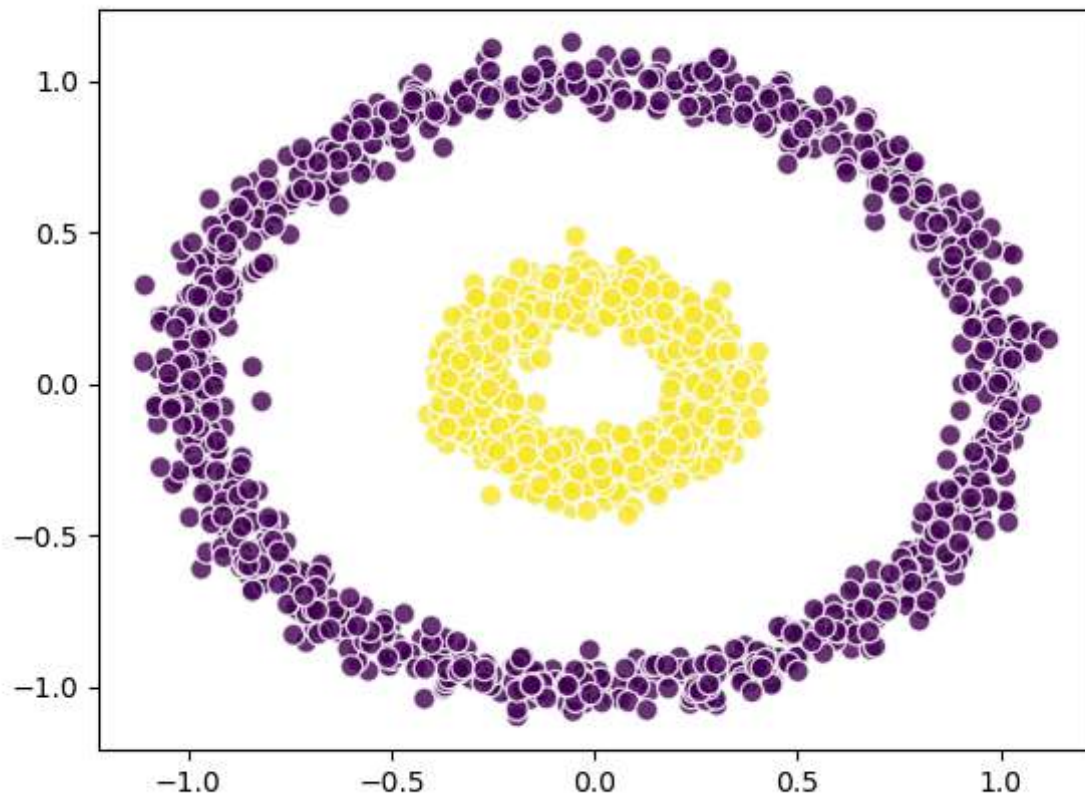
Python code:



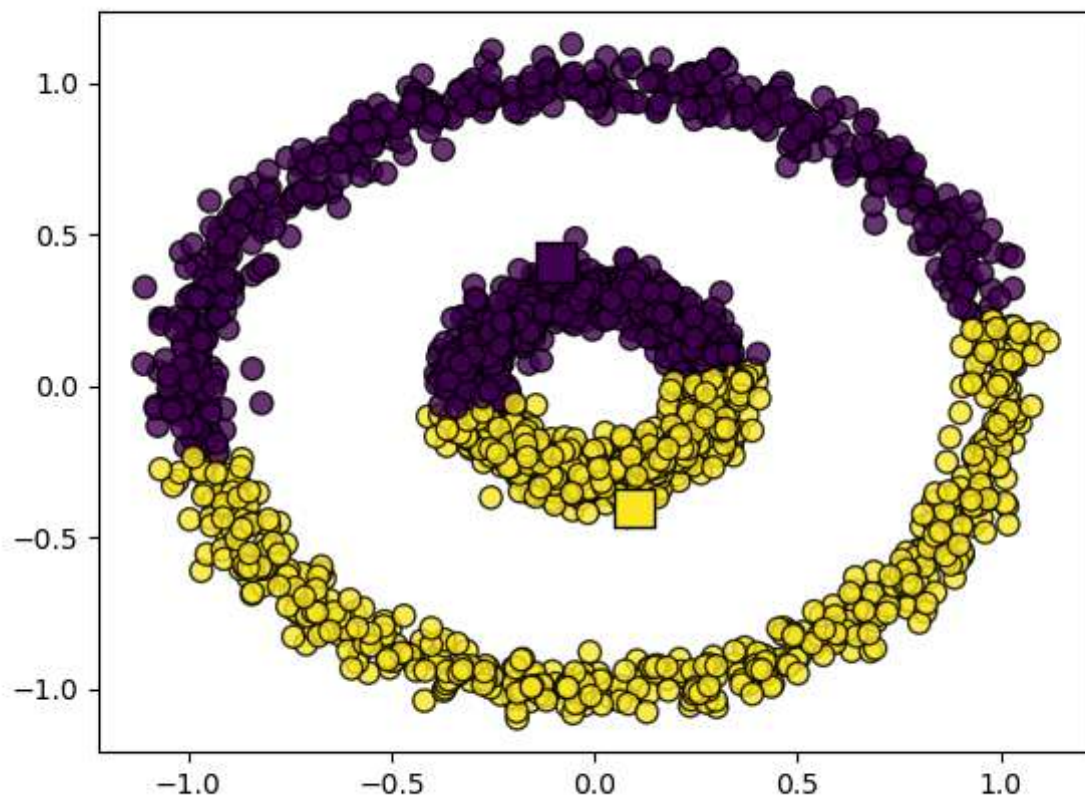
kmeans_blobs.py

Example 2

The second example illustrates a scenario where k-means is limited in its ability to detect good clusters. Below is another synthetic dataset generated to resemble two circles; one encircling the other.



People could easily perceive the two naturally occurring circular-shaped clusters in the dataset: (1) a set of purple data points that form the larger circle, (2) another set of yellow data points that make up the smaller circle. Detecting the same clusters, unfortunately, proves to be quite challenging to k-means, as illustrated below.



Python code:



kmeans_circles.py

Limitations

The previous example underlines some limitations of the k-means clustering algorithm:

1. First, the model strives to produce clusters with **uniform sizes**, even when the actual clusters do not necessarily have similar sizes.
2. K-means also assumes that data points surrounding a centroid are constrained within a **spherical area**. Consequently, when the shape of the true clusters does not conform to such a spherical assumption, the model is likely to perform poorly, as perfectly demonstrated in dealing with the circular-shaped clusters.
3. Finally, the algorithm is also sensitive to the presence of **outliers** and noises in data. Consequently, applying outlier detection and removal before running k-means is sometimes recommended.

Further readings

Yse, D.L. 2019. A complete guide to K-Means clustering algorithm. KDNuggets.

([https://www.kdnuggets.com/2019/05/guide-k-means-clustering-](https://www.kdnuggets.com/2019/05/guide-k-means-clustering-algorithm.html#:~:text=K%2Dmeans%20allocates%20every%20data,centroid%20than%20any%20other%20centroid)

[algorithm.html#:~:text=K%2Dmeans%20allocates%20every%20data,centroid%20than%20any%20other%20centroid](https://www.kdnuggets.com/2019/05/guide-k-means-clustering-algorithm.html#:~:text=K%2Dmeans%20allocates%20every%20data,centroid%20than%20any%20other%20centroid))